COMP 3059 – Capstone Project I

Software Requirements Analysis and Design Assignment

This assignment is an overview to gather the software needs with requirements analysis and help to proceed with the design.

The requirements analysis helps to break down functional and non-functional requirements to a basic design view to provide a clear system development process framework. It involves various entities, including business, stakeholders and technology requirements.

The design is the activity following requirements specification and before programming. Software design usually involves problem solving and planning a software solution.

To work on this assignment you could use the references and a sample template given below. The sample template can be customised to suit the nature of your project.

Reference Readings/Example:
http://www.uacg.bg/filebank/acadstaff/userfiles/publ_bg_397_SDP_activities_and_steps.pdf

www.cse.msu.edu/~chengb/RE-491/Papers/SRSExample-webapp.doc

Source for this template:
www.tricity.wsu.edu/~mckinnon/cpts322/cpts322-srs-v1.doc

1.0 Introduction

The Introduction section provides an overview of the system using software requirements analysis and design for the scope of the system.

The ClassMate, Student Life Management App is designed to help students organize their academic activities, monitor grades, and stay on top of important deadlines.

This document applies software requirements analysis and design principles to define the functional structure, data models, and interaction workflows for the system.

1.1 Purpose

This document describes the high level software requirements for the system. It describes the what, not how, of the capabilities of the system for the intended audiences.

The purpose of this document is to describe the high-level software requirements and design decisions for the ClassMate system, a web-based student life management platform that will be implemented as a microservice application. It defines what the system must do to support students in organizing their courses, assessments, deadlines, and academic progress, without prescribing how these capabilities will be implemented at a detailed technical level.

This document is intended for instructors, project stakeholders, and the development team. It provides a shared understanding of the system's expected behaviour, including its functional features, non-functional qualities, constraints, and dependencies. The Software Requirements Analysis and Design described here will serve as the baseline for subsequent design refinement, implementation, testing, and evaluation of the ClassMate MVP within the planned microservices architecture.

1.2 Scope

This explains what the proposed system will and will not do. Describe relevant benefits, objectives and goals. The description of scope should be consistent with the Project Plan.

ClassMate system will function as a web-based academic management platform developed using a microservices architecture. The system will enable students to manually manage their courses, tasks, reminders, and grades in a centralized environment. It will support academic planning by providing organized views of upcoming tasks, performance summaries, and reminders, helping students maintain awareness of their academic responsibilities.

The system is not intended to provide real-time communication tools, social networking features, automated data imports from LMS (Learning Management System) platforms, predictive analytics, or financial planning functionalities. Its focus remains on academic organization and student-driven data entry.

The overall goal of the system is to enhance academic productivity, reduce the complexity of tracking deadlines across multiple courses, provide visibility into progress trends, and establish a scalable architectural foundation for potential future expansions.

2.0 System Overview

The System Overview section introduces the system context and design.

2.1 Project Perspective

The Project Perspective describes the context and origin of the system by defining whether the system is:

- a follow-on member of a system family
- a replacement for existing systems, or
- a new self-contained system.

The ClassMate system is a new, self-contained web-based academic management platform developed specifically for the students. It is not a member of an existing product family and does not replace any prior system. Instead, it introduces an independent solution designed to support academic organization, deadline tracking, and performance monitoring. Within the broader category of student productivity tools, ClassMate positions itself as an organizational and planning platform distinct from LMS systems such as Brightspace, Blackboard, or Canvas, which primarily serve instructional and administrative functions. ClassMate focuses exclusively on student-centered academic planning, manual grade tracking, and task management.

2.2 System Context

The System Context describes the resulting software within the business case, including strategic issues in which the system is involved or which it specifically addresses.

ClassMate will operate as a microservices web application that enables students to manage their academic responsibilities in a centralized environment. The system will allow users to manually enter courses, tasks, reminders, and grades, and will present this information through dashboards and progress summaries. The system addresses several key problems identified in the Project Vision, including difficulty tracking deadlines, lack of visibility into academic performance, and the need for a unified place to capture tasks, reminders, and grade information. ClassMate supports students' organizational needs without relying on external data imports. It exists as an independent tool that improves academic time management, task visibility, and performance awareness.

2.3 General Constraints

General Constraints identify any business or system constraints that will impact the manner in which the software is to be:

- specified
- designed
- implemented, or
- tested.

The development and operation of the ClassMate system must adhere to the following constraints:
The system will be implemented using a microservices architecture.

The platform will function as a web application and must be accessible through common desktop and mobile browsers.

All data entered into the system will be manually provided by the student; no automatic external data retrieval is permitted in this phase.

The system must store and process academic information securely and efficiently using the selected backend technologies, including Postgres and MongoDB.

Design, scope, and implementation must remain within the limits of the academic project timeline and available team resources.

2.4 Assumptions and Dependencies

List any assumptions that have been made during the initiation of the project. In addition, list any dependencies that may impact its success or the desired result.

2.4.1 Assumptions

1. **Students will manually enter all academic data**, including courses, tasks, reminders, and grades, because the system does not integrate with external LMS platforms at this stage.

2. **Users will have stable internet access** to interact with the ClassMate web application, since the platform is not designed for offline operation.

3. **All microservices will be deployed in a controlled environment** (e.g., local Docker setup) and will remain accessible to each other through predefined service endpoints.

4. **The system will operate with moderate academic data volume**, typical of student workloads (e.g., 5–7 courses per semester, up to 25 assessments per course). This ensures the selected storage solution is sufficient.

5. **User authentication for the MVP is simplified**, meaning complex enterprise-level authentication is out of scope.

6. **Browsers used by students will support modern web standards** (HTML5, CSS3, JavaScript ES6), ensuring compatibility with the UI.

7. **Development team will follow the microservices architecture**, ensuring each service remains independent, loosely coupled, and aligned with the domain boundaries defined earlier.

2.4.2 Dependencies

1. **Java microservices framework** (Spring Boot) — required to implement independent services for courses, tasks, reminders, and grades.

2. **Database systems**

   The following types of databases will be required:

   > **Relational database:** Postgres for structured academic data.
   > **Nonrelational database:** MongoDB for unstructured notes, reminders, or Redis caching.

   There is a risk that data storage and service provider fees (such as Supabase) could be incurred in the feature as the user base grows.

3. **Containerization platform** (Docker) required to run microservices in isolated and reproducible environments.

4. **Internal service communication** microservices depend on RESTful APIs for interacting with each other.

5. **Hosting environment** , local development server, or container orchestration (e.g., Docker Compose).

6. **Notification service dependencies**, such as scheduled jobs, cron tasks, or Spring Scheduler to generate reminders at the correct times.

7. **Browser-based UI layer** — the front-end depends on API availability and the stability of backend endpoints to load dashboard data, deadlines, and grade calculations.

8. **Team workflow tools** (GitHub, Trello, GBC submission system), which the project depends on for collaboration, version control, planning, and delivery.

3.1 Functional Requirements

This section describes specific features of the software project. If desired, some requirements may be specified in the use-case format and listed in the Use Cases Section.

Each feature in the system must have its own functional requirement.

- Introduction
- Inputs
- Processing
- Outputs

3.1.1 Course Management

Introduction

Allows users to add, view, edit, and remove courses within the system.

Inputs

- Course name
- Course code
- Instructor
- Credits
- User selection (add, edit, delete)

Processing

- Validate course information
- Store course data
- Update user's course list
- Remove course and all associated tasks/grades
- Recalculate GPA and averages after changes
- Generate reports

Outputs

- Updated course list
- Confirmation messages (added/updated/deleted)
- Updated GPA and summary information

3.1.2 Task Management

Introduction

Enables users to manage tasks such as assignments, quizzes, exams, and projects for each course.

Inputs

- Task title
- Due date
- Weight
- Allocated marks
- Task completion status
- Bonus status
- Priority selection
- User selection (add, edit, delete)

Processing

- Validate task details
- Add or update task under a specific course
- Remove task and related grade
- Recalculate course weight distribution
- Update course current and accumulated grade values

Outputs

- Updated list of tasks per course
- Recalculated course averages
- Confirmation messages for task operations

3.1.3 Reminder System

Introduction

Allows users to set reminders for tasks, study sessions, or important dates.

Inputs

- Reminder description
- Date and time
- Optional task or course linkage
- User selection (add/edit/delete)

Processing

- Validate date/time
- Save reminder
- Schedule notifications
- Remove or update reminders

Outputs

- List of upcoming reminders
- Reminder alerts/notifications
- Confirmation messages

3.1.4 Grade Tracking

Introduction

Allows users to enter grades for tasks and view updated course performance.

Inputs

- Earned marks
- Task selection
- Course selection

Processing

- Validate marks (cannot exceed allocated marks)
- Store grade for the selected task
- Recalculate:
    - Current grade
    - Accumulated grade
    - Points lost
    - Maximum possible final grade

Outputs

- Updated grade and performance metrics
- Confirmation of grade entry
- Updated course grade view

3.1.5 Course Report & GPA Calculation

Introduction

The system calculates and displays academic performance metrics for each course and overall GPA.

Inputs

- All tasks within a course
- All grades
- Task weights
- Course completion data

Processing

- Calculate:
    - Current percentage
    - Accumulated percentage
    - Weighted course grade
    - GPA conversion
    - Points lost and grade projections
- Aggregate course results to compute user GPA

Outputs

- Course-level grade summary
- Overall GPA
- Graphs/visual summaries (if implemented)
- Updated dashboard statistics

3.2 Use Cases

3.2.1 Use Case #1 Add Course

Actor: User (Student)
Goal: Add a new course to their profile
Trigger: User selects “Add Course”

Preconditions:

- User is authenticated.

Main Flow:

1. System displays the “Create Course” form.
2. User enters course name, code, instructor, and credit info.
3. User clicks Save.
4. System validates the data.
5. System creates a new Course object and associates it with the User.
6. System confirms success.

Alternative Flows:

- 4a. Missing or invalid data:
  System shows error message and returns user to the form.

Postconditions:

- Course is added to user’s course list.

3.2.2 Use Case #2 Remove Course

Actor: User
Goal: Delete an existing course

Preconditions:

- The course exists.
- Course may contain tasks and grades.

Main Flow:

1. User selects a course and clicks Delete Course.
2. System asks for confirmation.
3. User confirms.
4. System deletes the course and all associated tasks/grades.
5. System updates GPA-related values.

Alternative Flows:

- 2a. User cancels:
  No deletion occurs.

Postconditions:

- The course and its tasks/grades are removed from the user account.

3.2.3 Use Case #3 Add Task to Course

Actor: User
Goal: Add a task (assignment, exam, quiz, lab, study) to a course

Preconditions:

- Course exists.

Main Flow:

1. User selects a course.
2. User clicks Add Task.
3. System shows task form.
4. User enters title, due date, weight, status, and allocated marks.
5. User clicks Save.
6. System validates data.
7. System creates Task and adds it to the Course.
8. System updates course's total weight.
9. System confirms success.

Alternative Flows:

- 6a. Invalid weight or missing fields → System shows error.

Postconditions:

- Task is added and course grade calculations are updated.

3.2.4 Use Case #4 Enter Grade for a Task

Actor: User
Goal: Record the grade earned on a task

Preconditions:

- Task exists.
- Task belongs to a course.

Main Flow:

1. User opens a Task.
2. User clicks Add Grade.
3. System displays grade form (marks earned).
4. User enters grade and clicks Save.
5. System validates the marks (cannot exceed allocated).
6. System updates:
    - Task grade
    - Course accumulated percentage
    - Course current grade
    - User overall GPA / average
7. System confirms success.

Alternative Flows:

- 5a. Marks > allocated: System shows error.

Postconditions:

- Grade is stored and all calculations update.

3.2.5 Use Case #5 Calculate Course Averages

Actor: User
Goal: View current grade and accumulated percentage for a course

Preconditions:

- Course has tasks.
- Some tasks may or may not have grades.

Main Flow:

1. User selects a course.
2. System calculates:
    - Current %: grade out of completed task weights
    - Accumulated %: total earned marks / total course weight

    - Maximum possible final %: accounting for lost marks
3. System displays results.

Postconditions:

- Updated grade metrics are shown.

3.2.6 Use Case #6 Add Reminder

Actor: User
Goal: Add a reminder

Preconditions:

- User is authenticated.

Main Flow:

1. User selects Add Reminder.
2. System displays reminder form.
3. User enters description, date, time, and optional course/task association.
4. User clicks Save.
5. System stores the reminder.
6. System sets up notification schedule.

Alternative Flows:

- 4a. Invalid date/time → System shows error.

3.2.7 Use Case #7 Remove Task

Actor: User
Goal: Delete a task from a course

Preconditions:

- Task exists.

Main Flow:

1. User selects a task.
2. User chooses Delete Task.
3. System asks for confirmation.
4. User confirms deletion.
5. System deletes the task and removes the grade if present.
6. System recalculates course averages.

3.2.8 Use Case #8 Generate Overall GPA

Actor: User
Goal: View GPA and averages across all courses

Preconditions:

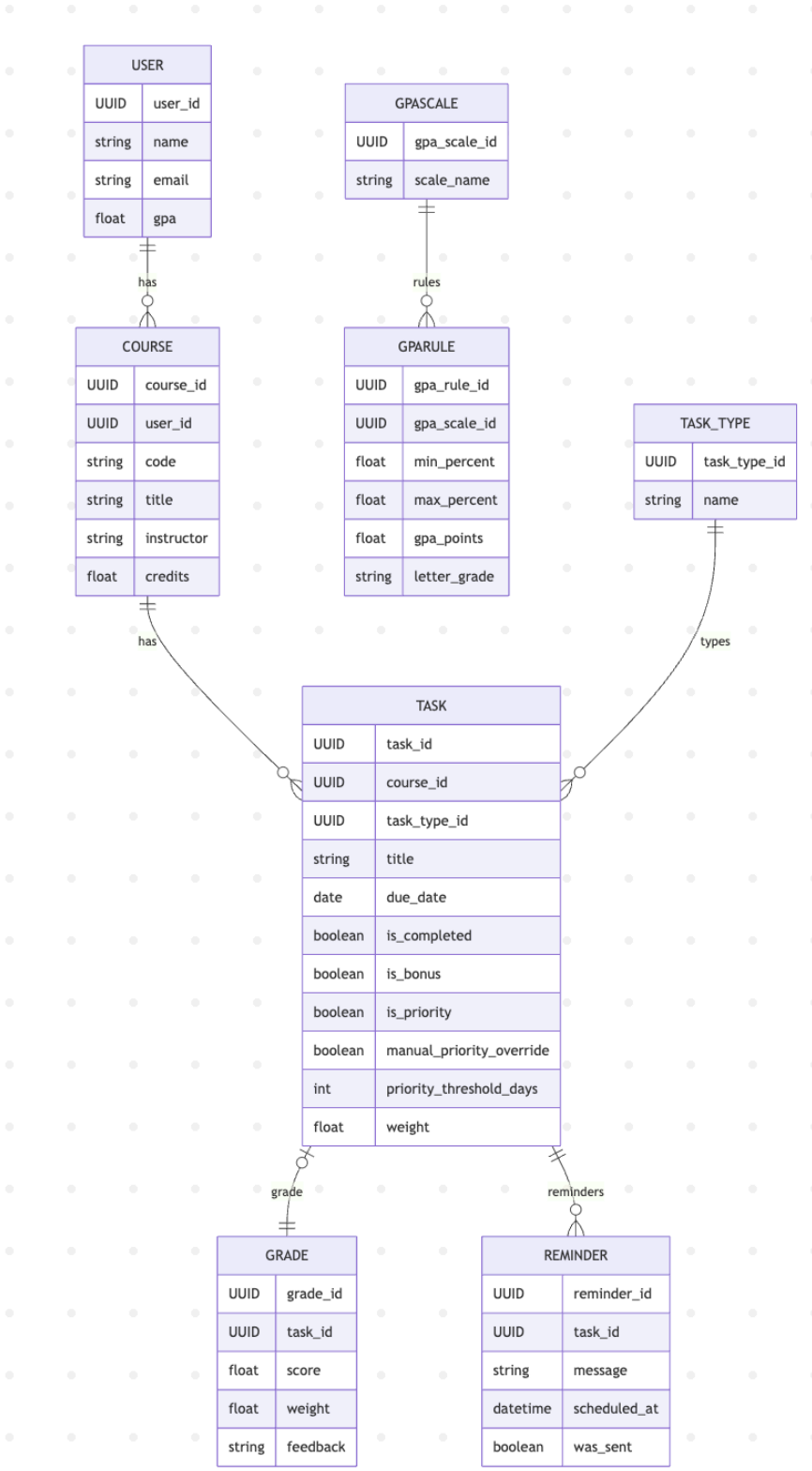
- User has at least one course with grades.

Main Flow:

1. User opens the Dashboard or Profile Summary.
2. System aggregates course-level grades.
3. System calculates GPA, total weighted average, and missing tasks.
4. System displays results.

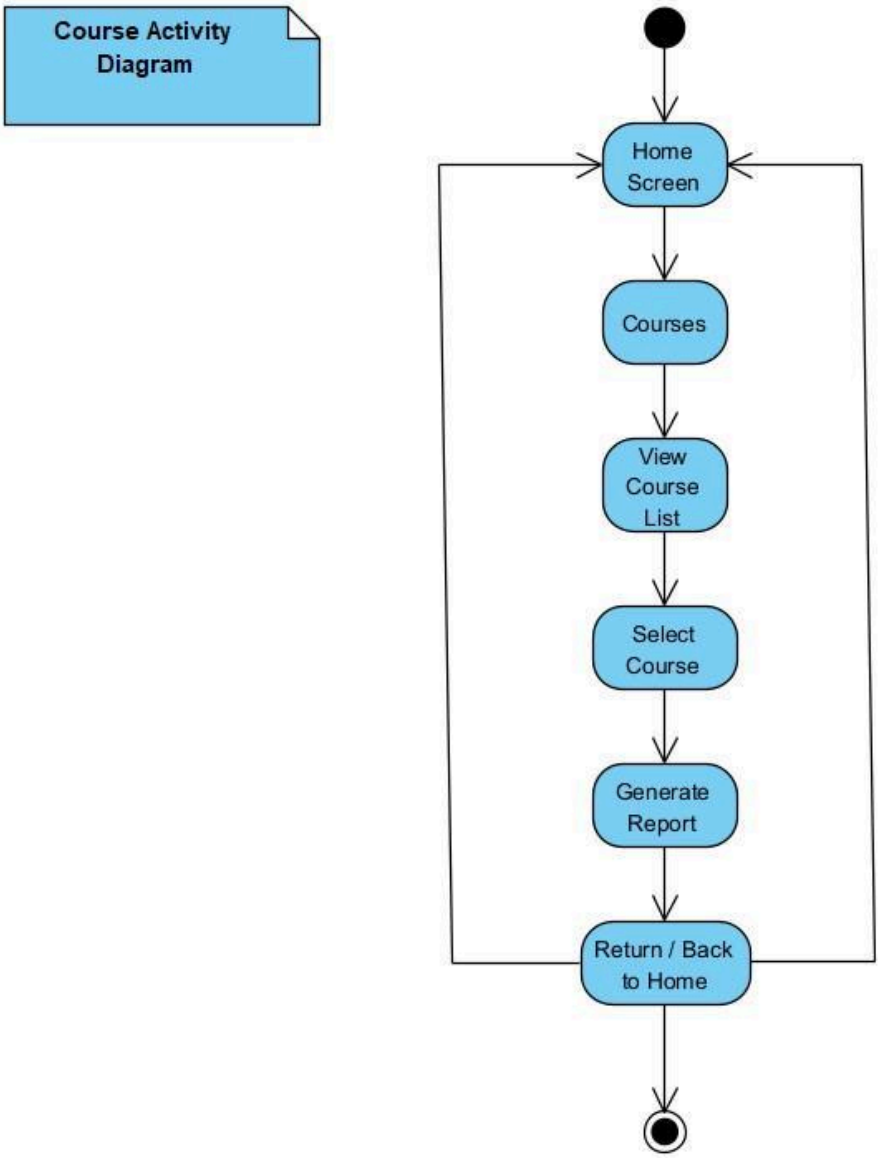
3.3 Data Modelling and Analysis

- Normalized Data Model Diagram

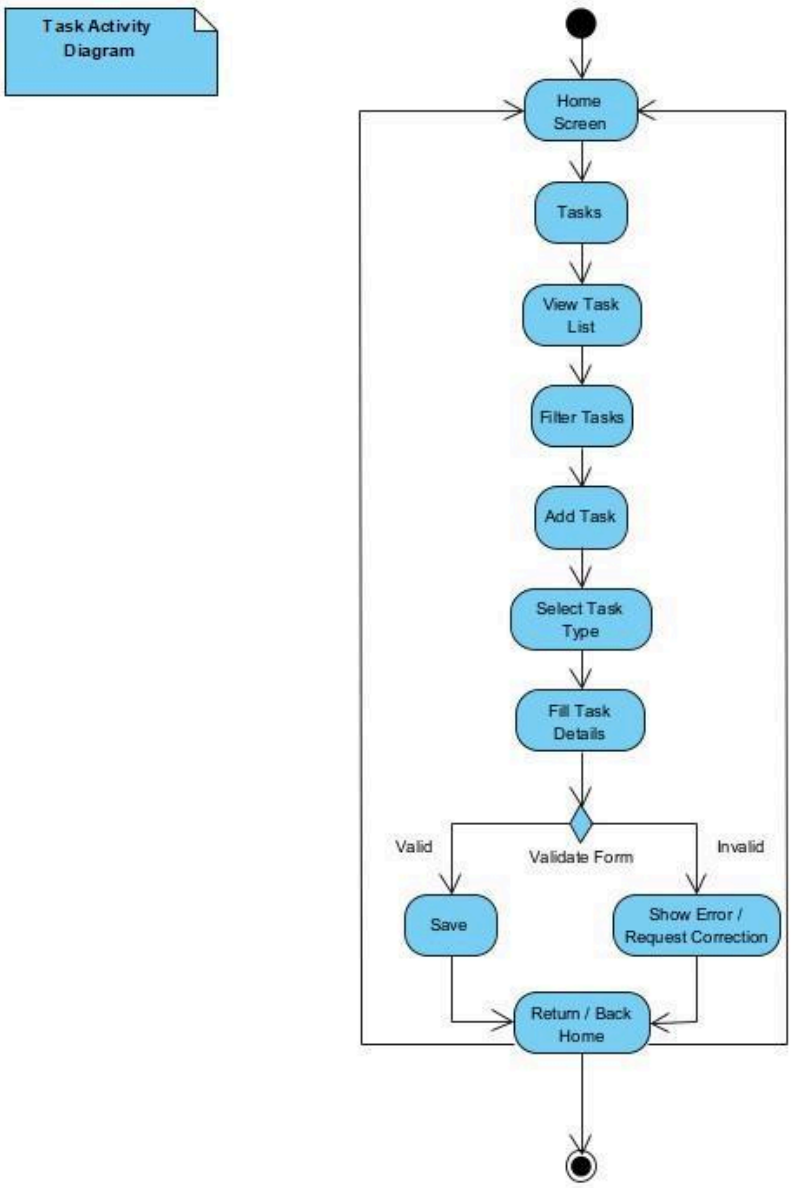


Activity Diagrams

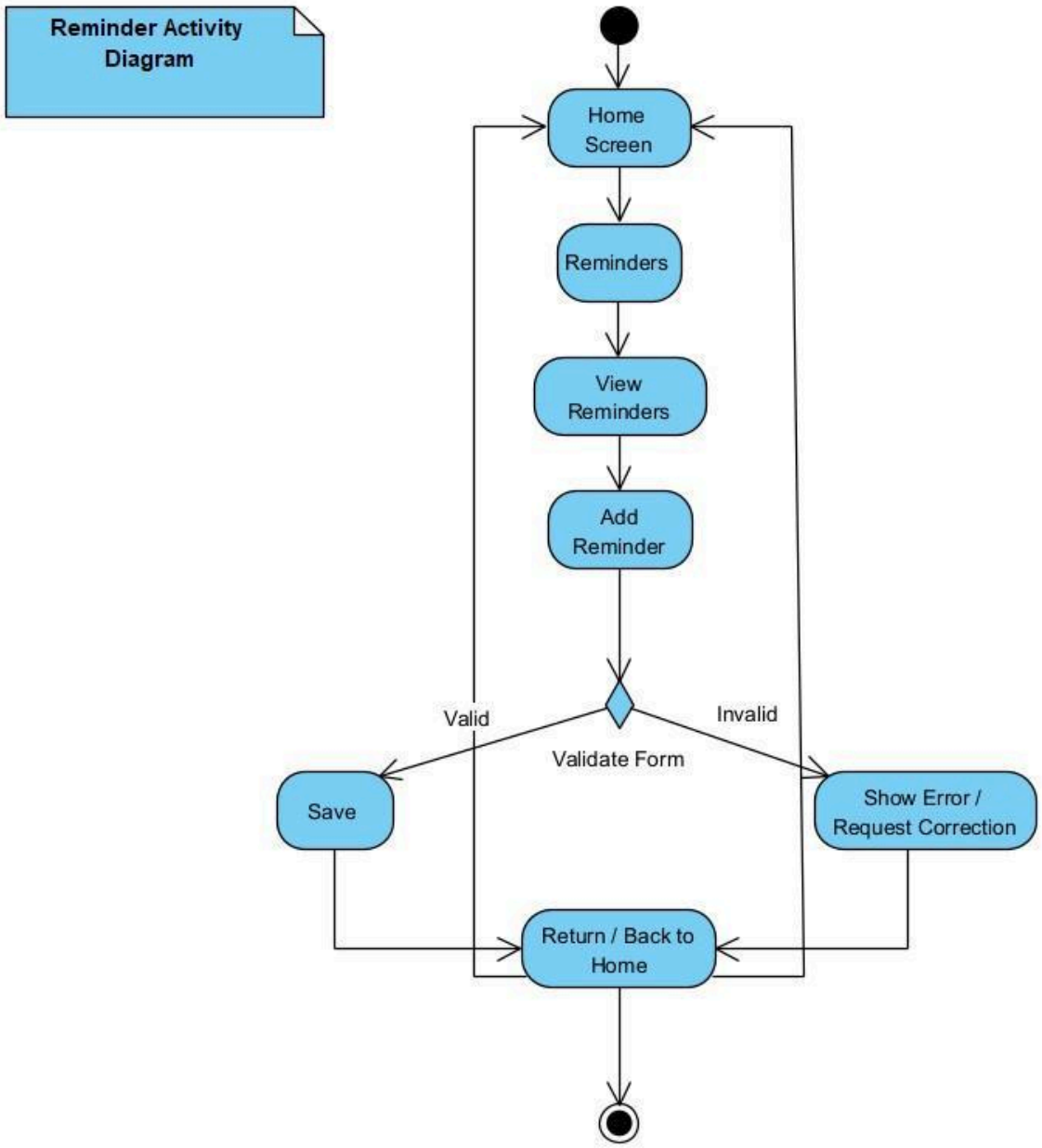
1. Course Activity Diagram:



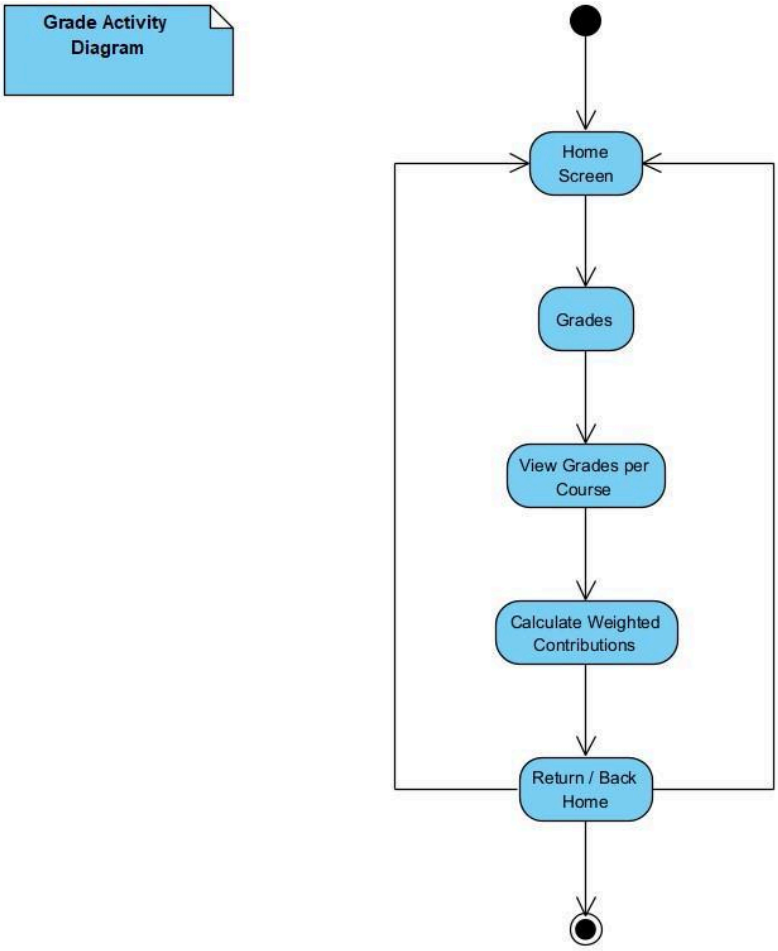
2. Task Activity Diagram



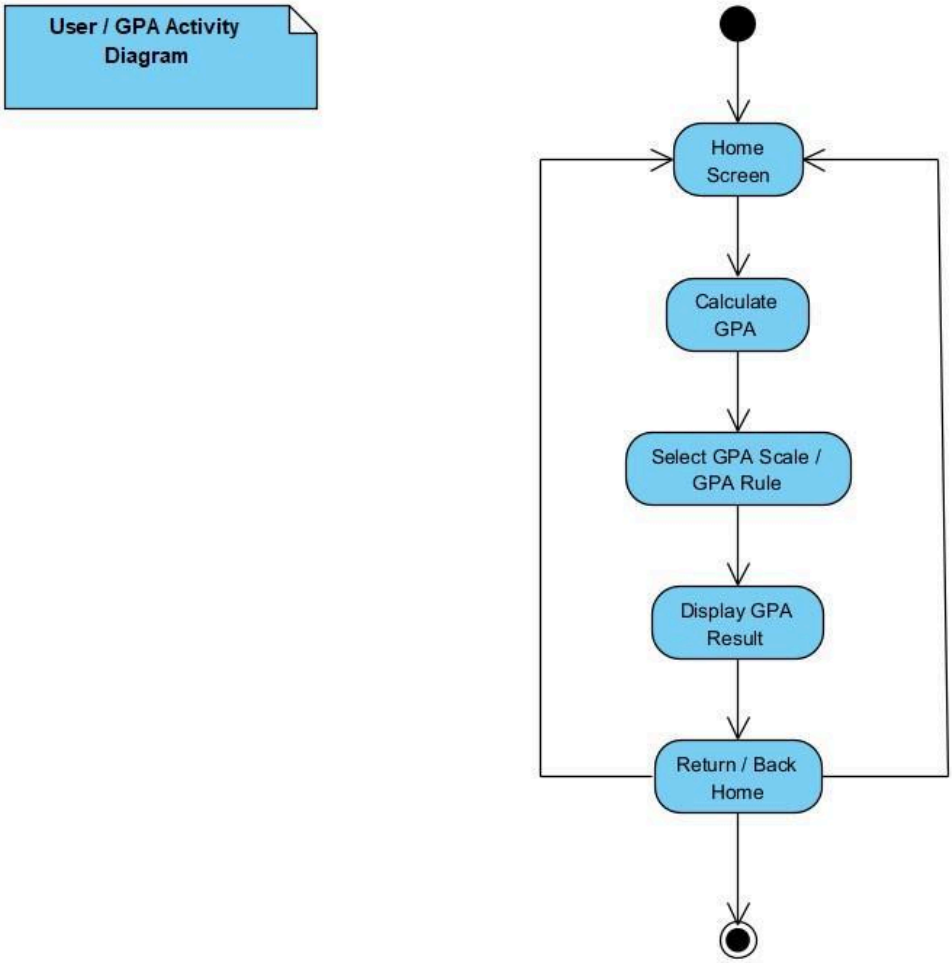
3. **Reminder Activity Diagram:**



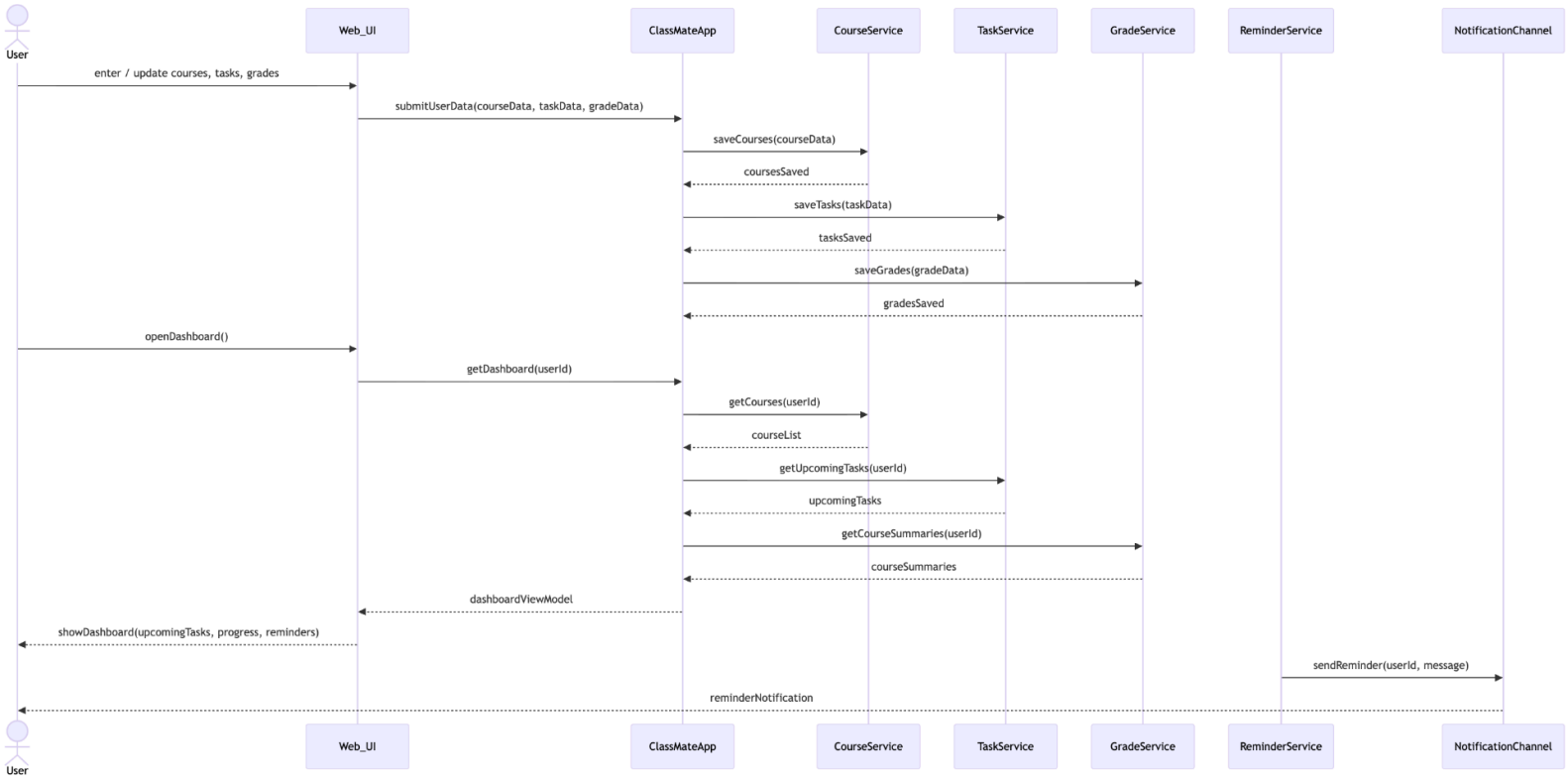
4. Grade Activity Diagram:



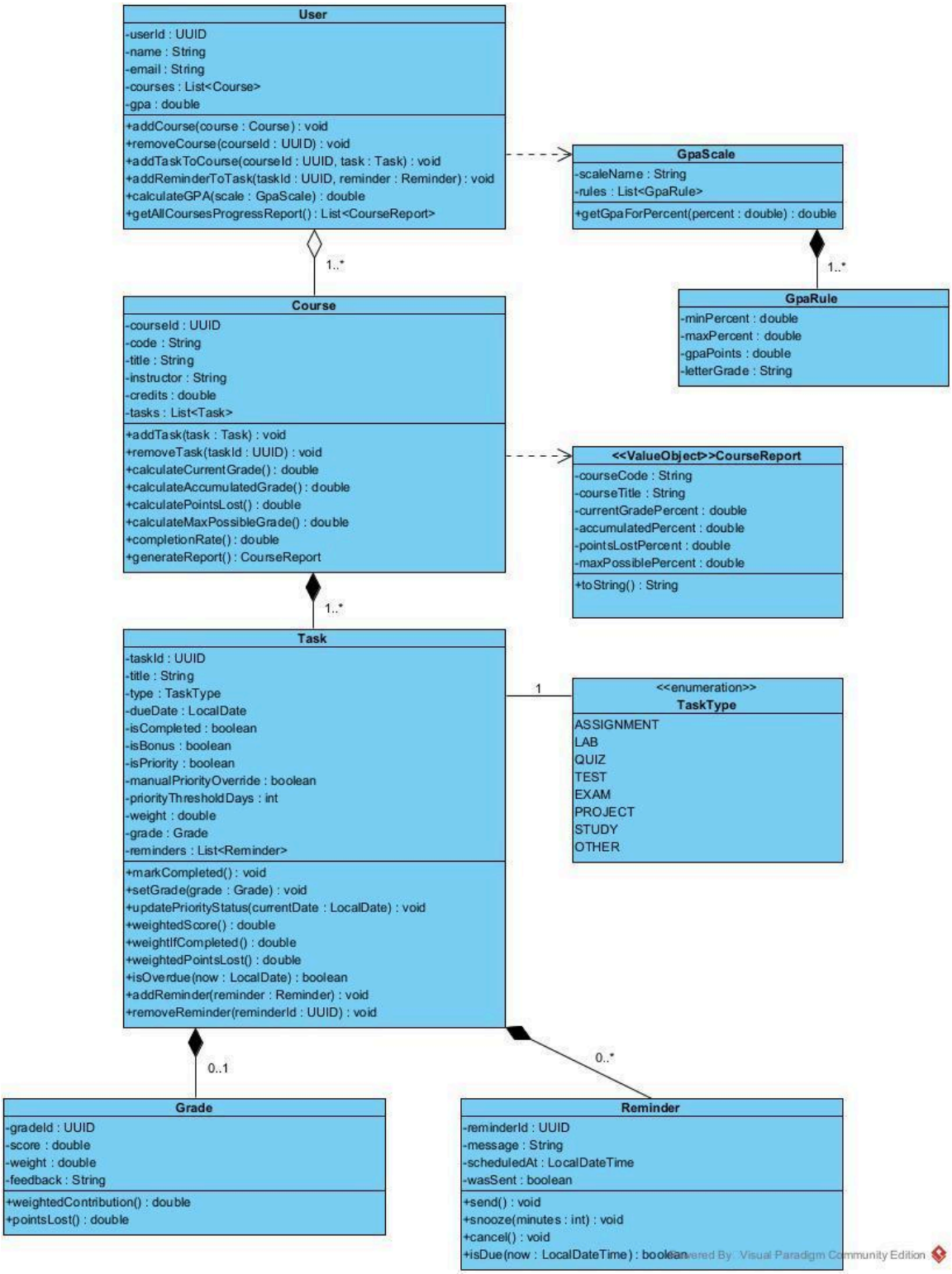
5. User / GPA Activity Diagram:



- Sequence Diagrams

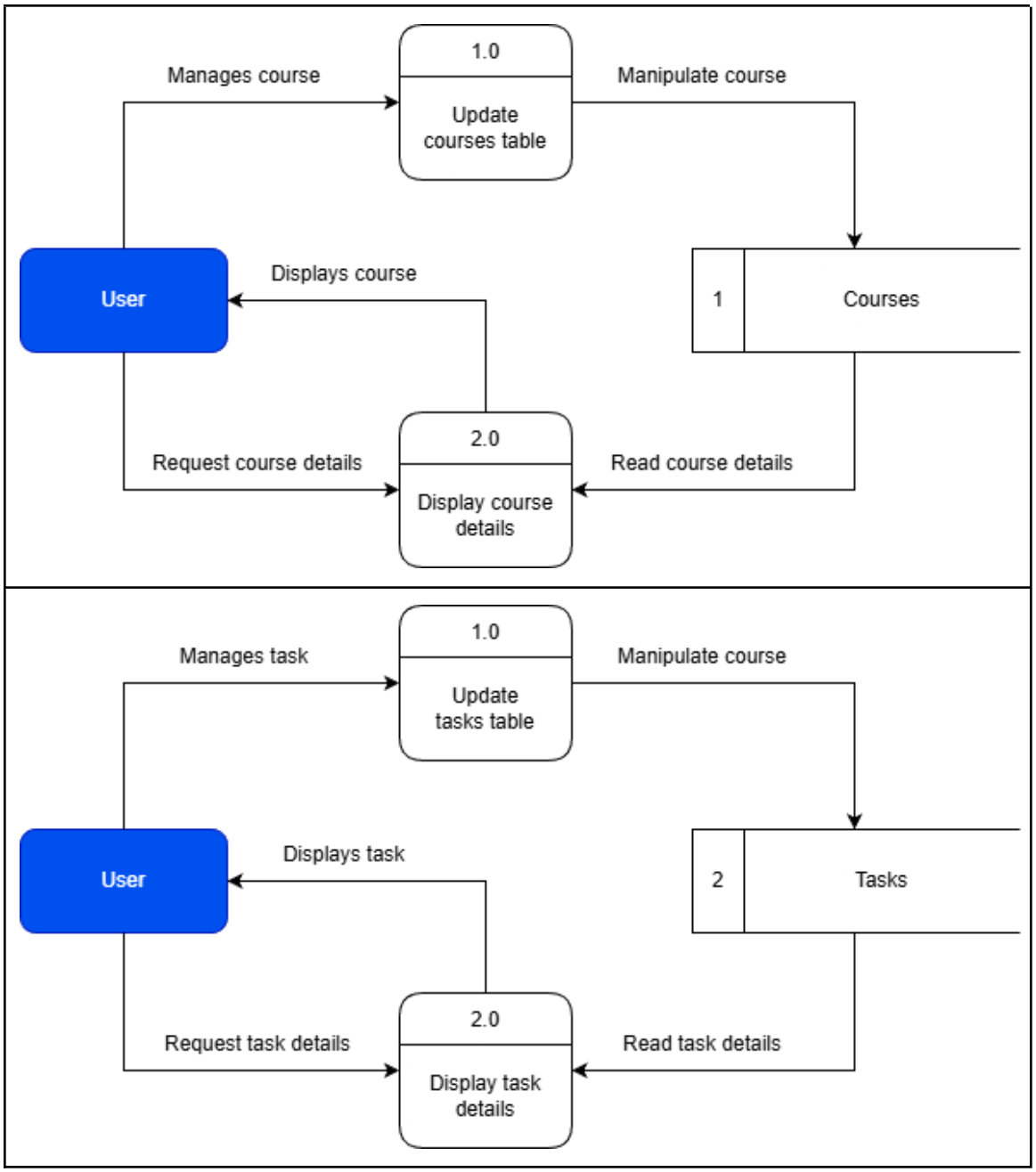


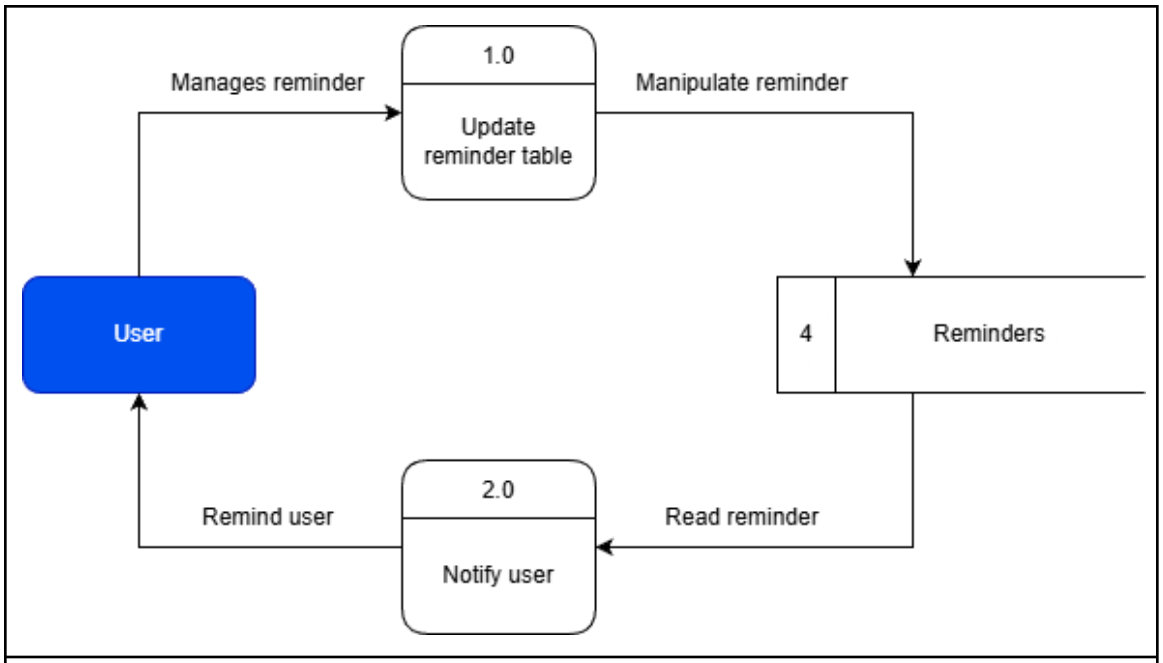
- UML Class Diagram

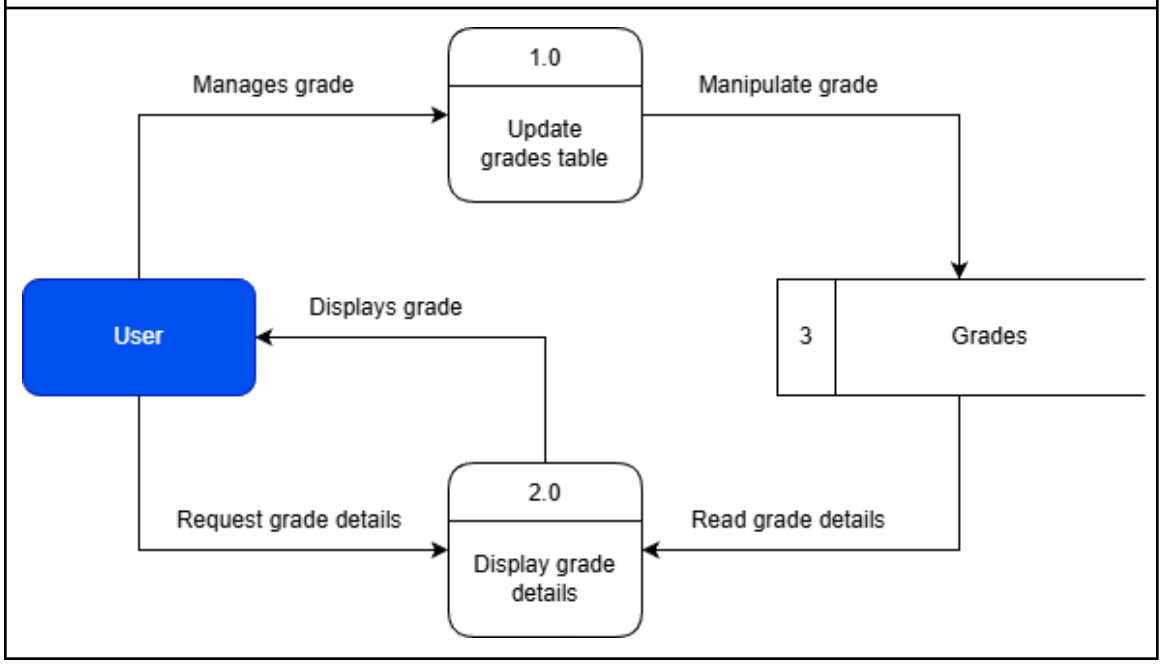


3.4 Process Modelling

- Data Flow Diagram




1.0
Manages reminder
Manipulate reminder
Update reminder table
User
4
Reminders
2.0
Remind user
Read reminder
Notify user


1.0
Manages grade
Manipulate grade
Update grades table
User
Displays grade
3
Grades
2.0
Request grade details
Read grade details
Display grade details

4.0 Non-Functional Requirements

The non-functional requirements for a system are typically constraints on the functional requirements – that is, not what the system does, but how it does it (e.g. how quickly, how efficiently, how easily from the user's perspective, etc.).

Non-functional requirements may exist for any of the following attributes – Performance, Reliability, Availability, Security, Maintainability, Portability.

Often these requirements must be achieved at a system-wide level rather than at a unit level. State the requirements in the following sections in measurable terms (e.g., 95% of transaction shall be processed in less than a second, system downtime may not exceed 1 minute per day, etc).

4.1 Performance

The system shall load the dashboard within 2 seconds for 95% of user sessions.

Course and task lists shall display within 1 second after user selection.

Grade calculations (current, accumulated, GPA) shall complete in under 0.5 seconds.

4.2 Reliability

The system shall correctly save and retrieve user data with 99% accuracy.

No more than 1% of scheduled reminders may fail due to application-level errors.

The system shall handle unexpected input without crashing.

4.3 Availability

The system shall be available for use 99% of the time, excluding planned maintenance.

4.4 Security

No sensitive academic data shall be transmitted without secure protocols (HTTPS).

Access to the application shall require user authentication.

Password hashing shall be utilized.

4.5 Maintainability

The system shall be modular so that updates to Course, Task, Grade, or Reminder modules can be made with minimal impact on other components.

New features shall be integrable with no more than 10% modification to existing code.

The codebase shall follow standard naming conventions and include documentation for major classes and methods.

4.6 Portability

The application shall operate on any modern web browser that supports JavaScript, HTML5, and CSS3.

The system shall adapt to multiple screen resolutions (desktop, tablet, mobile) with responsive layouts.

Migrating the application to another hosting environment should require minimal reconfiguration of environment variables and database connections.

5.0 Logical Database Requirements

Will a database be used? If so, what logical requirements exist for data formats, storage capabilities, data retention, data integrity, etc?

User data will be stored in Postgres and MongoDB using BaaS providers. The following logical requirements exist:

Data formats: Storing of dates will use a consistent format across database systems. This is to ensure the use of dates across the application can be seamlessly integrated with minimal conversions. All other fields will store textual data by default unless the nature of the data dictates another format. For example, grades will be stored as floating point numbers.

Storage capabilities: For the initial launch cost-effectiveness will be the priority to keep costs down. We will act to minimize the data stored for each user and only retain data that is necessary.

Data retention: User data will be stored for as long as accounts remain active. Data for inactive accounts will be deleted to reduce unnecessary storage use. Users can request to have their personal data safely deleted at any time.

Scalability: Despite the cost-effective focus, our data storage will be designed to be scalable as expansion is required. It will be designed for both horizontal and vertical scaling as data usage grows with our user base.

Data Integrity: Data validation will occur at the data layer to ensure all entries meet our business logic rules. This will ensure all stored data is complete and uniform and is fully usable throughout the application. This will also aid any audit, archiving, or application expansion that may be required in the future.

6.0 Other Requirements

Additional requirements, if any.

The system depends on the availability of the device's native notification services to deliver reminders and alerts. The application must be granted permission to access these services, and notifications may not function properly if the device's operating system restricts background processes or disables alerts. Reliable reminder functionality is therefore contingent on external platform-level notification support.

7.0 Approval

The signatures below indicate their approval of the contents of this document.

Project Role	Name	Signature	Date
Lead UI/UX Architect	Penny Ahlstrom		Nov 16, 2025
Lead Interaction & State Management Engineer	Nezihe Tekin		Nov 16, 2025
Lead Backend Engineer	James Jones		Nov 16, 2025
Lead Backend Architect	Omar McIntosh		Nov 16, 2025

w